

Autenticación Reactiva con Supabase y Arquitectura desacoplada

1. Introducción y Objetivos de la Sesión

Vamos a diseñar el control de acceso a la aplicación. Un error común es escribir el código de llamadas a la API de Supabase directamente dentro de las funciones de los botones (`onPressed`) en la interfaz de usuario. Esto rompe el principio de responsabilidad única y dificulta la creación de pruebas unitarias (*Unit Testing*).

Implementaremos un **Patrón Repositorio** que abstraiga el origen de los datos y utilizaremos un enfoque de **Programación Reactiva** para que las pantallas de la app escuchen en tiempo real si el usuario tiene una sesión activa o no, redibujándose de manera automática.

2. Fundamentos Técnicos

Antes de codificar, es imperativo revisar tres conceptos clave del motor de Supabase:

A. Proveedor de Identidad (GoTrue)

Supabase gestiona la autenticación mediante un microservicio interno de código abierto llamado **GoTrue**. Cuando un usuario se registra o inicia sesión, GoTrue valida las credenciales contra la tabla interna de la base de datos PostgreSQL (`auth.users`) y emite un par de tokens criptográficos:

- **Access Token (JWT):** Un token firmado que viaja en la cabecera de cada petición a la base de datos para verificar quién es el usuario. Tiene una vida corta por motivos de seguridad.
- **Refresh Token:** Un token de larga duración almacenado localmente que permite a la app solicitar un nuevo *Access Token* de forma invisible en segundo plano, evitando pedir credenciales al usuario constantemente.

B. El Ciclo de Vida de la Sesión (`onAuthStateChange`)

El SDK de Supabase expone un flujo asíncrono continuo (**Stream**) que notifica cualquier cambio en el estado de autenticación. Este flujo captura eventos críticos como:

- `INITIAL_SESSION`: Cuando la app arranca y verifica si ya existía un token guardado.
 - `SIGNED_IN`: El usuario ingresó sus credenciales con éxito.
 - `SIGNED_OUT`: El usuario destruyó la sesión de forma explícita.
 - `TOKEN_REFRESHED`: El token expiró y se renovó automáticamente en segundo plano.
-

3. Preparación en la Consola Web de Supabase (5 min)

Por defecto, Supabase tiene activada la **Confirmación por Correo Electrónico**. Esto significa que si un alumno registra un usuario en el laboratorio, no podrá iniciar sesión hasta que abra el correo y verifique la cuenta. Para agilizar la dinámica de la clase, desactivaremos esta opción:

1. Ingresa a la consola de tu proyecto en supabase.com.
2. En el menú lateral izquierdo, ve a **Authentication** (icono de candado o usuario).
3. Selecciona **Providers** y haz clic en **Email**.
4. Desactiva la opción **Confirm email** (Confirmar correo electrónico).
5. Haz clic en **Save** (Guardar).

4. Laboratorio Código Fuente Paso a Paso (45 min)

Organicen la estructura de carpetas de su proyecto bajo un enfoque limpio (Clean Architecture simplificada):

```
lib/  
├── domain/  
│   └── auth_repository.dart  
├── infrastructure/  
│   └── supabase_auth_repository.dart  
├── presentation/  
│   ├── auth_router.dart  
│   ├── login_screen.dart  
│   └── home_screen.dart  
└── main.dart
```

Paso 1: Definir el Contrato en la Capa de Dominio

Creamos una clase abstracta pura. Esta define *qué* hace nuestra aplicación, sin importar si los datos vienen de Supabase, Firebase o un servidor propio.

`lib/domain/auth_repository.dart`

```
import 'dart:async';

abstract class AuthRepository {
  /// Stream que emite el ID del usuario en tiempo real.
  /// Si emite `null`, significa que no hay una sesión activa.
  Stream<String?> get onAuthStateChanged;

  /// Registra un nuevo usuario en la base de datos. Retorna el UUID único.
  Future<String> signUpWithEmail({required String email, required String password});

  /// Autentica a un usuario existente. Retorna el UUID único.
  Future<String> signInWithEmail({required String email, required String password});

  /// Destruye la sesión actual local y remotamente.
  Future<void> signOut();
}
```

Paso 2: Implementar el Contrato con el SDK de Supabase

Aquí escribimos la lógica específica que traduce los métodos abstractos a comandos del SDK de Supabase.

lib/infrastructure/supabase_auth_repository.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';
import '../domain/auth_repository.dart';

class SupabaseAuthRepository implements AuthRepository {
  // Instancia única del cliente de Supabase
  final SupabaseClient _supabaseClient = Supabase.instance.client;

  @override
  Stream<String?> get onAuthStateChanged {
    // Escuchamos el stream nativo de Supabase y mapeamos su evento al ID del usuario
    return _supabaseClient.auth.onAuthStateChange.map((data) {
      final Session? currentSession = data.session;
      return currentSession?.user.id; // Retorna String? (UUID de Postgres)
    });
  }

  @override
  Future<String> signUpWithEmail({required String email, required String password}) async {
    try {
      final AuthResponse response = await _supabaseClient.auth.signUp(
        email: email,
        password: password,
      );
    }
  }
}
```

```

    );

    if (response.user == null) throw Exception('El registro no devolvió un
usuario.');
```

```

    return response.user!.id; // Retorna el UUID asignado por PostgreSQL
  } on AuthException catch (e) {
    // Captura errores específicos de Supabase (ej: Contraseña muy corta)
    throw Exception(e.message);
  } catch (e) {
    throw Exception('Error inesperado: $e');
  }
}

@override
Future<String> signInWithEmail({required String email, required String
password}) async {
  try {
    final AuthResponse response = await _supabaseClient.auth.signInWithPassword(
      email: email,
      password: password,
    );

    if (response.user == null) throw Exception('Credenciales incorrectas.');
```

```

    return response.user!.id;
  } on AuthException catch (e) {
    // Captura errores de login (ej: Usuario no encontrado, contraseña inválida)
    throw Exception(e.message);
  }
}

@override
Future<void> signOut() async {
  await _supabaseClient.auth.signOut();
}
}

```

Paso 3: El Enrutador Reactivo de Vistas (Middleware)

Esta pantalla no tiene diseño visual. Su única función es actuar como un árbitro que observa el **Stream** del repositorio y decide qué pantalla mostrar.

`lib/presentation/auth_router.dart`

```

import 'package:flutter/material';
import '../domain/auth_repository.dart';
import 'home_screen.dart';
import 'login_screen.dart';

class AuthRouter extends StatelessWidget {
  final AuthRepository authRepository;

```

```

const AuthRouter({super.key, required this.authRepository});

@override
Widget build(BuildContext context) {
  return StreamBuilder<String?>(
    stream: authRepository.onAuthStateChanged,
    builder: (context, snapshot) {
      // Mientras lee el almacenamiento local buscando tokens previos, muestra
      un loading
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Scaffold(
          body: Center(child: CircularProgressIndicator(color:
Colors.greenAccent)),
        );
      }

      // Si el stream emite datos (un UUID válido), el usuario está autenticado
      if (snapshot.hasData && snapshot.data != null) {
        return HomeScreen(authRepository: authRepository, userId:
snapshot.data!);
      }

      // De lo contrario (null o error), forzar el login
      return LoginScreen(authRepository: authRepository);
    },
  );
}

```

Paso 4: Interfaz de Login y Registro

Pantalla básica con validaciones de campos y manejo de loading controlado por estados.

`lib/presentation/login_screen.dart`

```

import 'package:flutter/material';
import '../domain/auth_repository.dart';

class LoginScreen extends StatefulWidget {
  final AuthRepository authRepository;
  const LoginScreen({super.key, required this.authRepository});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _isLoading = false;

```

```
void _executeAuthAction(Future<String> Function() authMethod) async {
  // Validar entradas básicas antes de disparar peticiones a la nube
  if (_emailController.text.isEmpty || _passwordController.text.isEmpty) {
    _showSnackBar('Por favor rellena todos los campos.', Colors.orange);
    return;
  }

  setState(() => _isLoading = true);
  try {
    await authMethod();
    // IMPORTANTE: No usamos Navigator.push aquí.
    // El AuthRouter detectará el cambio automáticamente a través del Stream.
  } catch (e) {
    _showSnackBar(e.toString().replaceAll('Exception: ', ''), Colors.redAccent);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}

void _showSnackBar(String text, Color color) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(text), backgroundColor: color),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Acceso al Sistema')),
    body: Padding(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          TextField(
            controller: _emailController,
            decoration: const InputDecoration(labelText: 'Correo Institucional',
border: OutlineInputBorder()),
            keyboardType: TextInputType.emailAddress,
          ),
          const SizedBox(height: 16),
          TextField(
            controller: _passwordController,
            decoration: const InputDecoration(labelText: 'Contraseña', border:
OutlineInputBorder()),
            obscureText: true,
          ),
          const SizedBox(height: 24),
          if (_isLoading)
            const CircularProgressIndicator(color: Colors.greenAccent)
          else ...[
            ElevatedButton(
              style: ElevatedButton.styleFrom(backgroundColor:
```

```

Colors.greenAccent, foregroundColor: Colors.white),
      onPressed: () => _executeAuthAction(() =>
widget.authRepository.signInWithEmail(
      email: _emailController.text.trim(),
      password: _passwordController.text.trim(),
    )),
      child: const SizedBox(width: double.infinity, child: Text('Iniciar
Sesión', textAlign: center)),
    ),
    const SizedBox(height: 8),
    TextButton(
      onPressed: () => _executeAuthAction(() =>
widget.authRepository.signUpWithEmail(
      email: _emailController.text.trim(),
      password: _passwordController.text.trim(),
    )),
      child: const Text('¿No tienes cuenta? Regístrate aquí'),
    ),
  ],
),
),
);
}
}

```

Paso 5: Pantalla de Éxito (Home Screen)

Muestra la información de la sesión recuperada directamente de la base de datos relacional.

`lib/presentation/home_screen.dart`

```

import 'package:flutter/material';
import '../domain/auth_repository.dart';

class HomeScreen extends StatelessWidget {
  final AuthRepository authRepository;
  final String userId;

  const HomeScreen({super.key, required this.authRepository, required
this.userId});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Dashboard Protegido'),
        actions: [
          IconButton(
            icon: const Icon(Icons.exit_to_app, color: Colors.red),
            onPressed: () => authRepository.signOut(),

```

```

    ),
  ],
),
body: Center(
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Icon(Icons.lock_open, size: 72, color: Colors.greenAccent),
        const SizedBox(height: 16),
        const Text('Sesión Validada en PostgreSQL', style:
TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
        const SizedBox(height: 12),
        Text('Tu ID Único de Usuario (UUID):', style: TextStyle(color:
Colors.grey[600])),
        const SizedBox(height: 4),
        Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(color: Colors.grey[200], borderRadius:
BorderRadius.circular(8)),
          child: SelectableText(
            userId,
            style: const TextStyle(fontFamily: 'monospace', color:
Colors.teal, fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ),
        ),
      ],
    ),
  ),
);
}

```

Paso 6: Ajustar el Punto de Entrada Global

Finalmente, inyectamos el repositorio concreto en la inicialización dentro del archivo `main.dart`.

`lib/main.dart`

```

import 'package:flutter/material';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'domain/auth_repository.dart';
import 'infrastructure/supabase_auth_repository.dart';
import 'presentation/auth_router.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

```



```
// 1. Inicializar la conexión física con el clúster de Supabase
await Supabase.initialize(
  url: 'https://TU_PROYECTO_ID.supabase.co',
  anonKey: 'TU_ANON_PUBLIC_KEY',
);

// 2. Instanciar nuestra implementación concreta
final AuthRepository authRepository = SupabaseAuthRepository();

runApp(MyApp(authRepository: authRepository));
}

class MyApp extends StatelessWidget {
  final AuthRepository authRepository;
  const MyApp({super.key, required this.authRepository});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.greenAccent),
      home: AuthRouter(authRepository: authRepository), // Inyección de
dependencia al Router
    );
  }
}
```
